Project Title: Lunk Gymness Database

Website link: http://classwork.engr.oregonstate.edu:1342/

1. **Members: Carter Deal, Samuel Dressel**

   Team Name: Team 121

A) Overview.....1
B) Database Outline.....2
C) ERD Diagram.....4
D) Schema Diagram (2NF).....5
E) Sample Data.....5
Verbatim Feedback:.....7
Summarization of the Feedback:.....20
Actions based on feedback:.....23
Fixes in Feedback We Chose to Ignore:.....24

A) Overview

Lunk Gymness, a newly created gym, has not yet created a digital management system for its gym. So far, they have used analog systems, but that proved to be more demanding as the gym grew. It became difficult to track members on paper, keep track of equipment as more was bought, and handle the creation and scheduling of classes. The website that we are proposing seeks to solve this management problem, creating a backend database of Members, Trainers, Equipment, and Classes, and creating a framework to handle the growth of said entities as the gym inevitably increases in popularity. As a newly created gym, they currently have 100 active members and 10 trainers. The gym has dozens of different types of equipment and hosts classes every day. Classes have a capacity of 15 people. A database-driven website will record class schedules and popularity, equipment usage (for classes and members), and the number of members, as well as monitor the constraints of classes, such as only one trainer,

class capacity not being exceeded, and members not registering for the same class twice. Our database is 2NF.

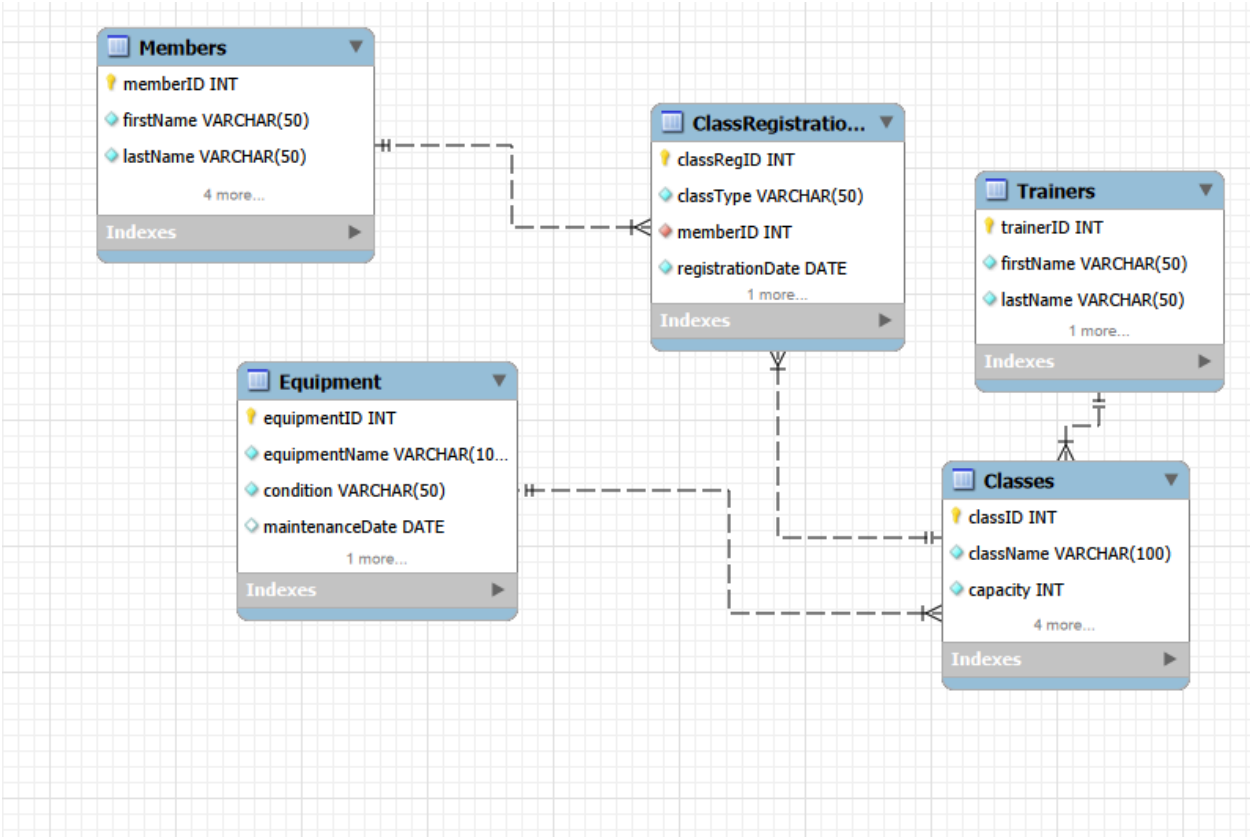
B) Database Outline

- Members: Stores personal and membership information for each gym member.
    - memberID: int, auto_increment, unique, not NULL, PK
    - firstName : varchar(50), not NULL
    - lastName: varchar(50), not NULL
    - email: varchar(100), unique, not NULL
    - phone: varchar(20), NULL
    - membershipType: varchar(20), not NULL (e.g., ‘Basic’, ‘Premium’)
    - joinDate: date, not NULL
    - Relationships:
        - M:M relationship with Classes via ClassRegistrations.
        - 1:M relationship with ClassRegistrations (one member can have many registrations)
- Trainers: Tracks trainers who lead classes and their areas of expertise
    - trainerID: int, auto_increment, unique, not NULL, PK
    - firstName: varchar(50), not NULL
    - lastName: varchar(50), not NULL
    - specialty: varchar(50), NULL (e.g., ‘Yoga’, ‘Strength’)
    - Relationships:
        - 1:M relationship with Classes (each class has one trainer)
- Classes: Stores details about each class, including schedule and assigned trainer.
    - classID: int, auto_increment, unique, not NULL, PK
    - className: varchar(100), not NULL

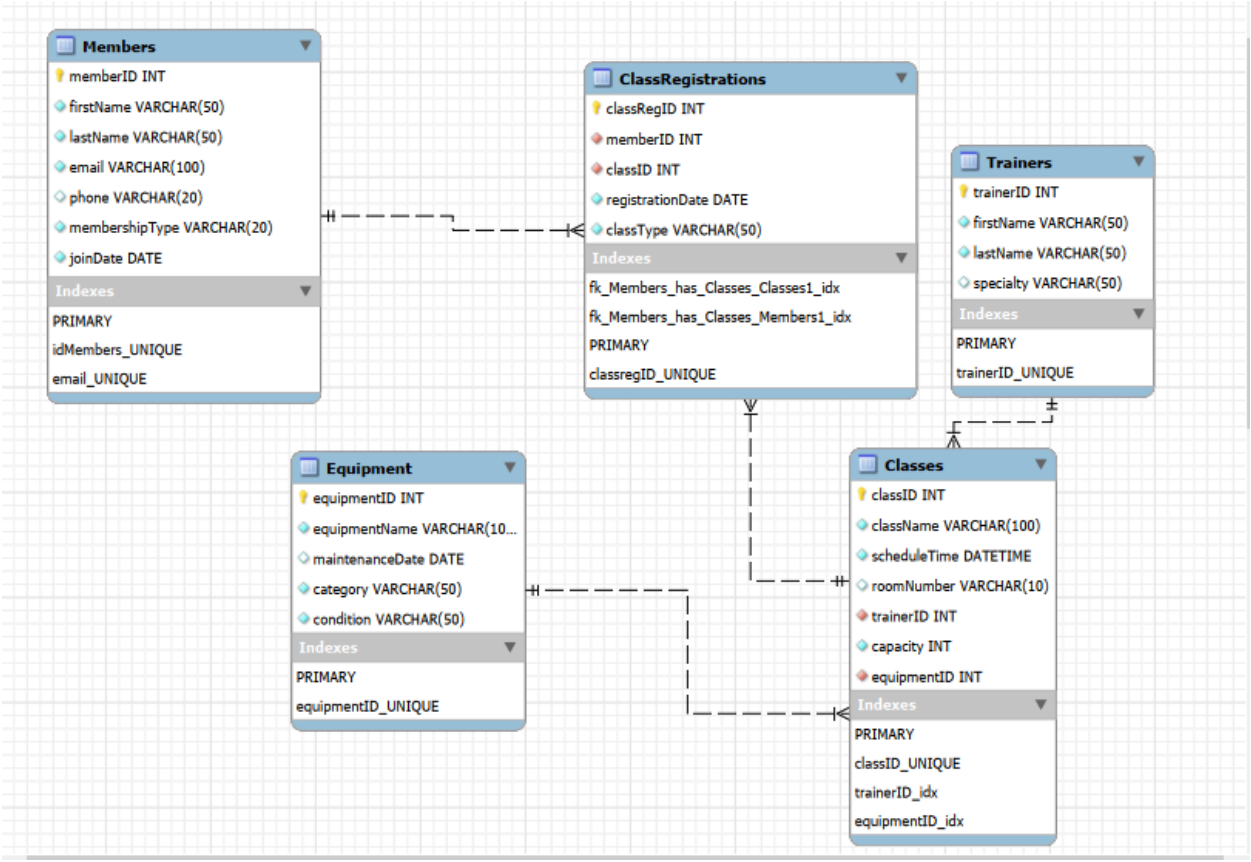
    - scheduleTime: datetime, not NULL
    - roomNumber: varchar(10), NULL
    - capacity: int, not NULL, (max capacity of 15)
    - trainerID: int, not NULL, FK -> Trainers(trainerID)
    - equipmentID: int, not NULL, FK-> Equipment(equipmentID)
    - Relationships:
        - M:1 relationship with Trainers implemented with trainerID as an FK in Classes
        - 1:M relationship with ClassRegistrations
        - M:M relationship with Members via ClassRegistrations.
        - M:1 relationship with Equipment with equipmentID as an FK classes
- Equipment: Maintains an inventory of equipment and its condition for scheduling and maintenance
    - equipmentID: int, auto_increment, unique, not NULL, PK
    - equipmentName: varchar(100), not NULL
    - category: varchar(50), not NULL (e.g., ‘Cardio’, ‘Strength’
    - maintenanceDate: date, NULL
    - condition: varchar(50), not NULL (e.g., ‘Good’, ‘Needs Repair’)
    - Relationships:
        - 1:M relationship with Classes with equipmentID as an FK in classes
- ClassRegistrations: Intersection table that implements the many-to-many relationship between Members and Classes. Tracks which members are registered for which classes.
    - classRegID: int, auto_increment, unique, not NULL, PK
    - memberID: int, not NULL, FK -> Members(memberID)
    - classID: int, not NULL, FK -> Classes(classID)

- registrationDate: date, not NULL
- classType: varchar(50), not NULL
- Relationships:
  - M:1 relationship with Members with memberID as an FK in ClassRegistrations
  - M:1 relationship with Classes with classID as an FK in ClassRegistrations
  - M:M relationship between Members and Classes (each row represents one member enrolled in one class)

C) ERD Diagram



D) Schema Diagram (2NF)



E) Sample Data

<u>Members</u>

memberID	firstName	lastName	email	phone	membershipType	joinDate
1	Sarah	Kim	sarah.kim@example.com	990-555-2011	Premium	2024-01-15
2	Jason	Patel	jason.patel@example.com	991-555-8890	Basic	2024-02-10
3	Maria	Lopez	maria.lopez@example.com	992-555-4421	Premium	2024-03-05
4	Daniel	Brooks	daniel.brooks@example.com	NULL	Basic	2024-04-01

Trainers

trainerID	firstName	lastName	specialty
1	Emily	Chen	Yoga
2	Marcus	Reed	Strength
3	Olivia	Turner	Cardio

Classes

classID	className	scheduleTime	roomNumber	capacity	trainerID	equipmentID
1	Morning Yoga	2024-05-01 08:00:00	R101	15	1	1
2	HIIT Blast	2024-05-01 10:00:00	R202	12	3	2
3	Strength Training	2024-05-02 09:30:00	R303	15	2	3
4	Evening Yoga	2024-05-02 18:00:00	R101	15	1	1

ClassRegistrations

classregID	memberID	classID	registrationDate	classType
1	1	1	2024-04-20	Yoga
2	2	2	2024-04-22	Cardio
3	3	3	2024-04-25	Strength
4	1	4	2024-04-28	Yoga
5	4	2	2024-04-30	Cardio

Equipment

equipmentID	equipmentName	category	maintenanceDate	condition
1	Yoga Mats	Flexibility	2024-04-10	Good
2	Treadmill	Cardio	2024-03-20	Needs Repair
3	Dumbbells Set	Strength	2024-04-15	Good
4	Rowing Machine	Cardio	2024-02-28	Good

Verbatim Feedback:

1. Bryan Figueroa: “Hello, below is my review of your current draft.

- The draft clearly shows a solution to the problem presented. Your gym is faced with an issue of trying to maintain its database via analog methods, and is looking to remedy this with a database that will accommodate its ever-growing gym, including members, trainers, classes, and equipment.
- The overview does list specific facts regarding your solution. I see that there is a clear and concise explanation of the problem you are faced with in this gym, and how you plan to address it with the creation of this database. I personally like how this is a very real problem for gyms, especially during the beginning months of the year as they sign up new members frequently.
- There are four entities that are summarized here: Members, Trainers, Equipment, and Classes. Each one represents the core aspects of what your gym will need to create their database, and each is represented accurately.

- Each entity is also detailed very well, with all the pertinent information listed for each, which I believe is a good representation of a real world database for a gym.
- The 1:M and M:M relationships are present and I am able to follow them in a very logical fashion. The notation in your ERD is also correct and clear,
- The naming conventions are also correct, with entities being plural and attributes being singular. Capitalization is also used appropriately.

  Overall, this is a very good draft, and I like very much how a real-world problem is presented with clarity, with your draft proposing a seamless and efficient manner to resolve with a DB creation.

  All work on this review is my own."

2. Austin Kassman: "My name is Austin Kassman and this is my review for the Team 121 project proposal.

   The overview clearly explains the problem the system is meant to solve. Lunk Gymness has outgrown its analog tracking as membership and class offerings increase, and the need for a centralized database is well justified. Including concrete numbers (around 100 members, 10 trainers, daily classes, and varied equipment) helps ground the proposal. One improvement would be to state the main pain point more explicitly, such as difficulty tracking class participation or equipment usage as the gym scales.

   The facts included do a good job establishing scope. Knowing the size of the member base, number of trainers, and variety of equipment makes the proposed database feel realistic. This supports use cases like tracking class popularity and scheduling. A brief note on expected growth or peak class capacity could further clarify future demands on the system.

   The entity design is solid overall. Members, Trainers, Classes, Equipment, ClassRegistrations, and ClassEquipment each represent a single concept and map cleanly to tables. The use of intersection tables for both member registrations and class equipment is appropriate and follows course patterns.

Entity details are mostly clear and well structured. Attributes have reasonable data types and constraints, and primary keys are clearly identified. The biggest gap right now is precision and consistency. For example, it would help to explicitly define composite primary keys or uniqueness constraints in the intersection tables and to standardize how foreign key references are described.
The relationships are logically sound. One-to-many relationships like Trainers to Classes make sense, and the many-to-many relationships are correctly resolved through ClassRegistrations and ClassEquipment. The ERD presents a coherent structure. One thing I would double-check is whether all listed relationships are necessary or if any could overlap conceptually, especially as the system grows.
Naming conventions are consistent throughout the proposal. Entities are plural, attributes are singular, and capitalization is used consistently, which makes the schema easier to read and reason about.
This review is entirely my own analysis."

3. Collin Vassallo: "Hello Group 121!

   Your overview well describes what the problem is that's being solved with your website and database. Gym membership with a scale of 100+ active members is an issue that is necessarily dealt with your proposal.

   Your overview also lists good facts about how the gym handled before, and will use a digital management system with numerical figures. I would suggest adding what constraints you could anticipate within the overview.

   The database's entities are well outlined and populated. They each represent a singular idea about how the gym expects to functions between classes, trainers, equipment, and members. I think that the ClassEquipment entity could be included within the Classes entity itself for simplicity, but it will certainly work as is.

   The outline does detail the purposes of the datatypes and relationships pretty well, such as including examples and how relationships will be implemented.

The relationships of 1:M and M:M are proper within the outline and ERD. There is a Many to Many relationship between Classes-Equipment and Classes-Members. The outline has a logical flow to read with, having Classes be in the top left, and it's leaves branching out through the rest of the diagram.

Namings are consistent in the outline throughout as camelCase. The only tiny discrepancy is Member's LastName within the EDR.

This response is all my own, unassisted work."

4. Carlos Chirinos: 'Hello Group 121. Below is my feedback on your draft.

Overview and problem statement.

The overview and problem statement are clearly stated in this draft that group is trying to solve. Which is that gyms are currently relying on analog systems, which are becoming inefficient as memberships, trainers, equipment, and daily classes are growing. So the project propose a database backed website that is intended to centralize the management of members, trainer, equipment, and class scheduling, which is able to directly answers scalability and origination issues that are being faced by the gym.

Specific Facts and Scope

Including the facts help illustrate the scope and scale. Providing that the gym has approximately 100 active members, 10 trainers, daily classes, and dozens of equipment types.This shows the need for a structured relational database and shows that the gym needs to be able to support frequent transactions, many-to-many relationships, and ongoing growth.

Entities and Conceptual Clarity

There are four entities that are each clearly defined, and each represents a single well described concept stored as a list. The core entities (Members, Trainers, Classes, Equipment) are all correct and align well with how they should be. The intersection

tables (like ClassRegistrations and ClassEquipment) are all correctly introduced to resolve many-to-many relationships, which shows the solid understanding of relational design principles.

Entity Details, Attributes and Relationships

Every entity has a clear purpose and has well chosen attributes, and appropriate data types and constraints. The primary keys are identified, and the foreign keys are used correctly to model the relationships. The outline explains the relationships in text, which makes it easy to follow. A minor improvement could be explicitly stating composite primary keys for intersection tables to prevent duplicate rows.

Relationships and ERD Logic

The design correctly includes both 1:M and M:M relationships. An example is the one-to-many relationship between Trainers and Classes, and the many-to-many relationships between Members and Classes and also between Classes and Equipment. The ERD correctly shows a logical and normalized view of the database that supports scheduling, enrollment tracking , and equipment usage without creating duplicates.

Consistency and Naming

The naming conventions are consistent across the overview, entity descriptions, and ERD. Entities are plural, attributes are singular, and capitalization is used correctly. Attribute names are clearly describing their contents creating clarity and maintainability. Making sure that all names match exactly between the outline and diagram (including capitalization) would further improve consistency

This review is all my own work."

5.Instructor/TA (Summarizing conversations): I(Samuel Dressel) asked why we got points off of our first draft and the TA and instructor mentioned that we were missing the project title. Instructor posted comments indicating that there are too many entities and suggested to remove

the classequipment table. Suggested to add a PK attribute, registrationdate, and classtype to the classregistrations table.

6.Michael Curry's feedback on Step 1 Final Draft:

Dear Team, good start on your project, this looks like a good idea for a RDBMS. Here is some feedback to help you be successful. The CS340 Project Guide: INTRODUCTION TO DATABASES (CS_340_400_W2026) requires at least 4 entities and at least 4 relationships, one of which must be a many-to-many relationship. You have two M:N and 6 entities, In order to allow time for the other assignments in the course, I recommend simplifying your design, by eliminating class equipment.

Also your intersection table is not properly designed. You need a PK for this table. Although composite keys are common they are not considered a best practice and we want you to avoid using them in this class. I also recommend that you also add the following attributes to classRegistration ID (PK as mentioned), registrationDate, classType (e.g. cardio, spin, etc. -this can be an Enum data type)

7.Hayden Barnes:

Hello Group 121!
The physical schema does indeed follow the outline and the ERD. Naming is consistent in pluralization and capitalization among attributes and entities. The schema is ordered and easy to read for the most part. It was difficult for me to see the primary and foreign keys, though this may be an experience issue on my part. The intersection tables are correct and follow best practices. I was unable to find any issues. The SQL syntax is correct and proper steps are taken for error handling within the database. The datatypes are correct and the varchar lengths are appropriate. The primary and foreign keys are properly defined and CASCADE operations are declared. The relationship tables within the PDF and ERD are consistent with what was declared in the SQL document. The sample data is also present in both the SQL file and the given PDF. The SQL is structured well with plenty of human made comments to describe each operation.

Good work!

This is my original work

8.Aspen Frazee:

Hello Carter and Samuel,

Lunk Gymness is a delight, I honestly cannot stop laughing. Cheers.

Onto the review:

Your ERD, Schema, and Outline all match in table and attribute names and relationships. A few notes: following camelCase styling, the attribute classregID should be classRegID.

Your schema is very easy to read! I understood perfectly what the relationships are between the tables. One thing to note: 9.1.4.1 Adding Foreign Key Relationships Using an EER Diagram defines the difference between the solid or dotted lines used to connect tables. Be sure that your line choices are congruent with identifying/non-identifying relationships.

Your intersection table is good! I especially appreciate that the intersection table has its own PK.

From what I can tell, your normalization looks good! I was getting a bit tripped up on a potential 3NF issue in Classes between roomNumber and capacity, but I think I'm overcomplicating things.

I tried running your SQL in PhPMyAdmin but got this error message:

"#1142 - CREATE command denied to user 'cs340_frazeee'@'localhost' for table `cs340_dressels.Equipment`"

I asked ChatGPT and TA Joy Kim, and both confirmed this was a privileges issue. Considering that your DB is "hardcoded into your file" (Joy's words), I decided to go through it manually, meaning I might miss a few errors.

> Note: ChatGPT did say that USE cs340_dressels; and cs340_dressels.TableName is redundant.

I would also agree with this assessment, as it would make it easier for future reviewers. Otherwise, you can include instructions on how they can lightly modify the SQL to make it easier to import.

The commenting on the code is a bit strange. There is a repeated comment at the top of your file.

```
-- -----------------------------------------------------
```

```
-- Schema cs340_dressels

-- -----------------------------------------------------




-- -----------------------------------------------------

-- Schema cs340_dressels

-- -----------------------------------------------------
```

I think some basic comments on the top and bottom statements could also be helpful, as they are pretty cryptic.
You do not define any ON DELETE or ON UPDATE cascades, which would be a good idea.
The SQL is mostly well structured, aside from my few notes above!
As a reviewer, I wrote almost all of this content. The only section ChatGPT-included opinion is marked above in the blockquote.

9.Alexander Lane:

Hey Carter and Samuel! Cool project idea, looks like solid trajectory.
In terms of relationships, I was wondering if it was logical that a class only has one piece of equipment? Aligns with the M:1 relationship defined in the outline still. The rest looked solid.
It looks like the schema model follows the outline and ER diagram exactly.
There is a consistency in the naming convention between the outline, ERD and schema model.
Entities plural, attributes singular with correct capitalization.
The schema is very easy to read, great work there.
The intersection table ClassRegistrations is properly formed with two FKs
I noticed in the outline ya'll mention the database is 2NF. As far as I could tell, the data looks correctly normalized to 2NF, great job! Honestly I didn't notice any transitive, except maybe the capacity for rooms? All good if the capacity is meant to be dependent on the class itself.

I tried out the SQL file and it looked great but didn't work on my end. The file seems tailored specifically for the cs340_dressels schema so it would've taken work on my end to have it work for me. So that probably impacts the quality of the rest of the review a bit.
The data types are appropriate given the descriptions. I was unfamiliar with the UNIQUE INDEX syntax but it seems reasonable. Wasn't able to verify for the previous reason.
PK and FK syntax looks good, the CASCADE operations are declared and seem appropriate (namely NO ACTION for UPDATE and DELETE on FKs)
The relationship tables are present when I tried out the SQL on my end.
All example data was inserted as far as I could tell. Nicely added under the respective table.
The SQL is well structured and commented, thank you for that! Made it a lot easier on my end.
Overall, great job so far and I'm excited for the rest of the project! Feel free to respond if anything was confusing.
All original work.
10.Michael Curry: Dear team, I have the following comments. In your normalization process you should have recognized the potential issues that could happen from insert and delete anomalies and take steps to prevent these in your DDL. Instead of hardcoding all your example data please consider using subqueries as you did in Intermediate SQL Assignment (on Gradescope).
You should anticipate and prevent delete/update anomalies by using methods as described in Exploration - MySQL Cascade: INTRODUCTION TO DATABASES (CS_340_400_W2026)
The ERD is very sparse (maybe too much so)! Please see the following response to another student question about the ERD vs Schema as well as this page in Canvas ERD vs Schema: INTRODUCTION TO DATABASES (CS_340_400_W2026). According to the outline, "record class schedules and popularity, equipment usage (for classes and members), and the number of members, as well as monitor the constraints of classes, such as only one trainer, class capacity not being exceeded, and members not registering for the same class twice" So I think that you should help your managers better understand your plan by adding the className and equipmentName to your ERD. Maybe you can think about if there are any other attributes that would be insightful.
Great job overall improving from the last step, I think you can easily address these changes in your Step 3. I look forward to seeing your project develop.
11.Michael Curry: The draft UI for your project looks good! A couple of suggestions for you to consider, Don't see an ERD in your PDF. The ERD and Schema you have are identical. This is not part of the rubric for this draft, but you will lose points in the final

w/o an ERD. Please see the following ERD vs Schema: INTRODUCTION TO DATABASES (CS_340_400_W2026).
Also you have already implemented some of the CRUD operations! This is admirable, but not what was instructed. Actually we will require the use of PL/SQL for CUD operations so you will end up having to change this in the future. Its unfortunate when work gets duplicated, but I hope you learned a lot about coding this phase and it was a positive experience.
Looking good, did you use AI tools to accelerate development? If so please be transparent and cite your work. Its also a good idea to say it was your own work if not. Please see Code Citation Tips: INTRODUCTION TO DATABASES (CS_340_400_W2026) for examples.

Also, it is generally a bad idea to use Select * in your DML, b/c it is common for the tables to be modified and this may then add unexpected columns to your UI which cause problems that you did not anticipate in the design.

12. Chase Schrepel:
Does the UI utilize a SELECT for every table in the schema? In other words, data from each table in the schema should be displayed on the UI. Note: it is generally not acceptable for just a single query to join all tables and displays them.
Yeah, its available in the DML.
Does the UI implement an INSERT form for at least one table in the schema? In other words, there should be UI input fields that correspond to at least one table.
Yes, it inserts into all of them, classes, trainers, registration, etc.
Does the UI have at least one DELETE for any one entity? In other words, is there a form/link/button that will allow the deletion of a row in at least one table?
There is a delete for all that need it yes.
Does the UI have at least one DELETE that will remove things from a M:M relationship? In other words, if an order is deleted from the Orders table, it should also delete the corresponding rows from the OrderDetails table, BUT it should not delete any Products or Customers.
Yes it does, but a more universal delete instead of directly a M:M one

Is there at least one UPDATE form in the UI for any one entity? In other words, in the case of Products, can productName, listPrice, qtyOnHand, e.g. be updated for a single ProductID record?

Yep! tons of updates where needed, good job.

Do you have any other suggestions for the team to help with their HTML UI? For example, using AS aliases to replace obscure column names such as fname with First Name.

It really looks good to me. Reading the comments of my peers I would say that following what Samantha said about possible delete anomalies would be a good thing to check out. Otherwise your website is clean. Great job.

This was all my work, not AI.

13.Jonathan Salfen:

Hey Samuel and Carter, this is looking great. Looks like your UI is very nearly complete. Great work!

Does the UI utilize a SELECT for every table in the schema? In other words, data from each table in the schema should be displayed on the UI. Note: it is generally not acceptable for just a single query to join all tables and displays them.

- Yes it does.

Does the UI implement an INSERT form for at least one table in the schema? In other words, there should be UI input fields that correspond to at least one table.

- Yes it does.

Does the UI have at least one DELETE for any one entity? In other words, is there a form/link/button that will allow the deletion of a row in at least one table?

- Yes it does.

Does the UI have at least one DELETE that will remove things from a M:M relationship? In other words, if an order is deleted from the Orders table, it should also delete the corresponding rows from the OrderDetails table, BUT it should not delete any Products or Customers.

- Yes, not just for a M:M relationship but for all entities.

Is there at least one UPDATE form in the UI for any one entity? In other words, in the case of Products, can productName, listPrice, qtyOnHand, e.g. be updated for a single ProductID record?

- Yes it does.

Is there at least one UPDATE form in the UI to modify an M:M relationship? In other words, does the UPDATE allow the user to select a different foreign key value to update the intersection table with?

- Yes, not just for a M:M relationship but for all entities.

Do you have any other suggestions for the team to help with their HTML UI? For example, using AS aliases to replace obscure column names such as fname with First Name.

- This UI looks pretty complete at this point. I don't have any constructive feedback. Great job!

This was all my work.

14. Benjamin Lynch:

Hey Carter & Samuel, I think you guys have a great idea for your project, and the UI is really clean! The Home Page, Nav Bar, and input fields are looking great.

SELECT is used for every table, with the DDL using SELECT *, instead of specifying what rows to bring; I could see this leading to future issues with mapping in case rows change, but otherwise everything looks good.

Delete and Edit actions are present for all entities, including the M:M ClassRegistrations. The Edit page also has a great layout, bringing the pre-existing information forward to the new page.

All these tables look really good, and I don't think I could change anything, besides maybe removing the trainerID/EquipmentID from the display of Classes, unless it is needed for anything on the frontend/UI side. Your UPDATE, DELETE and INSERT statements within DML.SQL look good, assuming the WHERE parameter is just passed the entity ID, and I think the CASCADE declarations are all good.

I have no other proposed changes, everything looks great!

All my original work.

15.Samantha Hoffman:

Overall your project looks like its coming along great so far. It implements select and insert as well as update from numerous tables, all that was required is met. Additionally the local impact of delete looks good and functional. My only concern is that you might be having to delete anomalies. When you delete all of the equipment used for a class that class is deleted as well, this seems flawed, I feel it should change to an error state somehow, or perhaps have a null value. This overall makes me think that equipment being non null for classes seems odd,

especially because of just how gyms work in general. I can see many classes saying bring your own, or fully provided, or none needed. I hope this helps and great work with everything!

16.Professor Curry:
Dear team, I am appreciate of your efforts. The UI shows a clear interface to the DB documented in the PDF, good work. However, there are a few issues that I hope you can address.

I don't see any of the NEW or UPDATE forms on your UI page. The buttons are there but I could not find a way to access any of them to verify the design. Perhaps they were there in Step 3 Draft b/c you had full CRUD implemented but got removed in Final? Again, it is unfortunate when teams get deducted points for not following instructions.

Good work updating your DDL to include the insert statements. Also good comments at the top of the DDL and DML. Did you use AI to help modify, see Code Citation Tips: INTRODUCTION TO DATABASES (CS_340_400_W2026)?

I can see you attempted to address my comment on the ERD vs Schema but it still needs work. What purpose does displaying the implementation detail of the intersection table serve for a business perspective in the ERD? W/o any attributes it is hard to justify imo. Perhaps you can either add classType or remove this from your ERD.

Good job adding the sample data to your UI.

I still do not see a citation in your work regarding originality. Is this based on the CS340 source code? Did you use AI to help modify, see Code Citation Tips: INTRODUCTION TO DATABASES (CS_340_400_W2026)

Additional notes. Per the Project Step 6 (Portfolio Assignment) check your planned operations versus this list. I believe you have more than required. That is ok, but get the requirements done first and then if you want to add extras you can.

- Minimum project requirements (written succinctly as possible):
    - 3 rows of sample data illustrating each table's function
    - 4 entities
        - Plus an intersection table. 5 tables total.
    - 4 relationships

        - Including 1 M:N relationship
    - Select/Read/Display - all tables
    - Create/Insert - 1 table, using PL/SQL
    - Update - 1 table, using PL/SQL
    - Update M:N - 1 M:N relationship
    - Delete - 1 table, using PL/SQL
    - Delete M:N - 1 M:N relationship
    - RESET DB, using PL/SQL

If you have any questions, please feel free to message me on Teams. I expect that with continued effort you can address these issues for the next project step and be reasonably successful on the final project.

17. Lucas Feldsien: Do all SELECT operations work as expected? In other words, is data from all the database tables displayed in the web application? If not, please describe which tables data is not being displayed from.

all operations look like they are working.

- Is a RESET button/link implemented that properly resets the database? If not, describe the behavior/what data is not being reset.

  RESET works, it undid any changes i made.
- For any additional CRUD operations that the team listed as operational, do they function as the team expects (e.g. if the team stated that a CRUD step worked, but you found an error, please tell them)?

  All the operations are working.
- Would a user easily be able to use the UI to complete the CRUD operations that the team listed as working? If not, please elaborate and provide helpful suggestions for how the UI can be improved.

  UI looks great no problems.

This review is my own work.

18. Collin Fugate: Hey Carter and Samuel, I looked through your zip file and here are my thoughts.
SELECT/Browse: All 5 tables (Equipment, Trainers, Classes, Members, ClassRegistrations) have browse pages and the SELECT queries in app.js use `SELECT *` which should pull everything from each table. One thing I noticed is that the Classes and ClassRegistrations browse pages just show raw foreign key IDs (like trainerID and equipmentID as numbers) instead of showing the actual trainer name or equipment name. It still works, but it might be harder for a user to know what those IDs mean. A JOIN in those queries would help with that when you get to later steps.
RESET: The reset button is in the navbar which is easy to find. The stored procedure drops all 5 tables and recreates them with sample data, so it should fully restore everything. One thing I wanted to flag — the reset procedure doesn't use SET FOREIGN_KEY_CHECKS=0 before dropping tables. Right now it works because you drop them in the right order (ClassRegistrations first, then Classes, etc), but if the order ever got mixed up it could fail. Not a big deal but something to keep in mind.
DELETE: The delete is done through a stored procedure (DeleteEntry) which takes in a table name and ID, thats a cool approach since you only need one procedure for all tables. The cascade deletes look like they're set up correctly in the DDL — Classes has ON DELETE CASCADE for both trainerID and equipmentID, and ClassRegistrations cascades on both memberID and classID. So deleting a trainer should cascade and remove their classes, and then those class deletions should cascade and remove the related registrations. That chain looks right.
UI stuff: The delete buttons have confirmation prompts which is nice so you don't accidentally delete something. One thing — on the home page (index.hbs) the Members card actually links to `/trainers` instead of `/members`. So clicking "Members" from the homepage takes you to the Trainers page which is probably just a copy-paste typo. Also the add/edit forms don't have POST routes set up in app.js yet (you mentioned insert and update aren't working yet) so those forms won't actually submit anything right now, but you already know that.
DML.sql: The DML file still has placeholder `WHERE criteria` in the DELETE and UPDATE queries instead of actual parameterized conditions. You'll probably want to update those to match what your app.js actually does (like `WHERE memberID = ?`) before the final submission.
Suggestions: When you get to implementing insert and update, you already have the form templates ready which is great. You'll just need to add the POST/PUT routes in app.js to handle the form submissions. Also for the add forms, some of them ask for the primary key ID (like classRegID or classID) as a manual input — since those are AUTO_INCREMENT in your DDL you probably don't need users to type those in, the database will handle it.
Overall good progress, the delete and reset are solid and the cascade behavior is set up correctly. Good luck with the remaining CRUD steps!
This review is all my own work. I read through the team's uploaded code files to write this feedback.

19. Anthony Giusti Shindelus: All SELECT operations work as expected, where all of the database tables are displayed in the web app.
The reset button has been properly implemented and resets every record of every table in the database to their original values.
Deleting values from tables works as expected, and the cascade effects are accurate, where deleting something like a class from the classes table will indeed delete any records that reference this class from the registrations table.
Overall the UI is intuitive, but records that are referenced via foreign key are only displayed as their id number, which makes it difficult to actually understand all the data, such as which member is actually associated with the registration or which trainer is assigned to a particular class.
Your app is coming along well, and I like the layout and styling.
All of this feedback is my own.

20. Nishat Yisman: **1. Do all SELECT operations work as expected? In other words, is data from all the database tables displayed in the web application? If not, please describe which tables data is not being displayed from.**

Yes, SELECT operations appear to be working as expected, with data from all database tables displayed in the web application.
2. Is a RESET button/link implemented that properly resets the database? If not, describe the behavior/what data is not being reset.

Yes, RESET works as expected. I tested this by deleting records from each table and then clicking the "Reset Database" button to confirm the database was restored successfully.
3. For any additional CRUD operations that the team listed as operational, do they function as the team expects (e.g. if the team stated that a CRUD step worked, but you found an error, please tell them)?

The DELETE operation is listed as working for every page in the UI. I tested this (as part of #2 above) and confirmed that it functions as expected. The delete cascades also worked correctly. For example, deleting a Trainer properly deletes any corresponding Class and ClassRegistration records referencing that Trainer. (I appreciate that you listed expected behavior for all deletion scenarios in your Ed post.)

While reviewing your DDL for the ON DELETE CASCADE constraints, I noticed that you also used ON UPDATE CASCADE. Are you sure these are necessary? They're typically only needed if you plan to update primary keys, which your UI currently doesn't allow and is generally uncommon practice.

4. Would a user easily be able to use the UI to complete the CRUD operations that the team listed as working? If not, please elaborate and provide helpful suggestions for how the UI can be improved.

Overall, yes. The UI is polished and easy to navigate, and you've made several usability choices that work really well! For example, since the Edit forms are separate pages (instead of being embedded in the View pages), I appreciate that you prefill the fields with data from the selected record. You also implemented confirmation popups for DELETE actions to prevent accidental deletions. Finally, I love that the "Reset Database" button doesn't redirect to the homepage; instead, you stay on the page where the change was made, which makes it easy (saves a click!) to verify that the reset worked.

I do have a few suggestions for improvement.

[1] Display foreign key names instead of IDs for readability (e.g., in the Classes and ClassRegistrations tables). You can accomplish this by using JOINs in your SELECT queries in the DML.

[2] Consider whether the "Category" attribute of the Equipment table should be its own entity (a category table). This is currently a text field in the Add and Edit forms, which introduces the possibility of typos and update anomalies. However, I noticed for the "Condition" attribute (another potential category table) that you avoid the need for a separate entity by restricting options via a dropdown. You could apply the same approach to "Category" to reduce inconsistencies.

Whichever direction you choose, I recommend briefly addressing this in your PDF (perhaps in the Schema section). You note at the end of your Overview that your database is in 2NF; however, explicitly discussing the decision not to move to 3NF would help show that this was a deliberate design choice.

[3] Similarly, consider whether the "Specialty" attribute of the Trainers table should be its own entity (a category table). Just like "Category", this is a text field in your Add and Edit forms, which carries the risk of typos and update anomalies. Based on your Sample Data, each trainer lists only one specialty. If trainers are limited to a single specialty, you could restrict options via a dropdown as a workaround to implementing a separate entity. If trainers can have multiple

specialties and you choose to keep the text field (rather than implement a Specialty category table with an intersection table [e.g., TrainerSpecialties]), it may be especially helpful to explain this decision in your PDF.
[4] Since you've already implemented confirmation popups for DELETE actions, you might consider doing the same for the RESET operation.
[5] I noticed that the Edit and Delete buttons are side-by-side on the Trainers page but stacked on all the other pages. You may want to choose one layout and apply it across all pages. I only mention this because the brightly colored buttons make the discrepancy fairly noticeable, but it's obviously a very minor suggestion! I understand that layout constraints (like maintaining consistent page width) probably influenced your decision, which is perfectly reasonable.
5. What suggestions do you have for the team in any areas where they are blocked or having difficulty? Detailed, helpful feedback will receive higher credit. If the team is not blocked or is having difficulty, encouraging and supportive comments would be a better response than NO feedback.

The team doesn't seem to be blocked currently, and they're steadily working on implementing the required INSERT and UPDATE operations. (They also mention working on "browse" functionality, but I'm not sure what this entails since their SELECT operations appear to be fully in place.)
Overall, excellent work and *phenomenal* styling. I could hardly believe this wasn't a React app. You've clearly gone above and beyond what was provided to us in the explorations re: styling. I could only find a few small opportunities for improvement, but you already have a very strong project on your hands. Best of luck finishing out the rest of the term!
(This review is entirely my own work.)

21. Professor Michael Curry: Great progress team! The DELETE and RESET worked for me on the equipment page, and I see that you addressed my prior comments about the ERD and the subqueries instead of hard coding FK IDs. The EDIT is not working though at this stage it isn't required, so I believe that you are on track and making good progress. You should easily be able to scale this up for the next project steps. Keep up the good work!

Summarization of the Feedback:

1. Bryan Figueroa: This was a very positive review that gave no suggestions for improvement. It detailed how the draft met all the requirements for the assignment, such as stating that it accurately describes a proposed problem, the overview lists specific facts, there are 4 entities presented, the entities are properly described, 1:M and M:M relationships are present and logical, and the naming conventions are satisfied.

2. Austin Kassman: A positive review as well, but gave some feedback on entity relationships and the overview/problem statement. Gave the suggestion to describe the problem in more detail. Mentioned that giving brief points on expected growth and class capacity would be beneficial. Also gave the suggestion of standardizing foreign key reference descriptions and clarifying composite primary keys.
3. Collin Vassallo: An overall positive review that detailed how the draft met all the requirements, gave a brief suggestion of naming anticipated constraints in the overview section. Pointed out that there was a small typo in the ERD where the lastName field was capitalized.

4. Carlos Chirinos: This post seemed very positive and supportive of our draft. The review said we met all of the criteria, but suggested one improvement. Carlos suggested that we state our primary keys for intersection tables clearly in order to not create duplicate rows.
5. From TAs/Instructor: They suggested that we add a project title, remove the classequipment table, and add a unique PK attribute, classtype, and registrationdate to the classregistrations table.

6. The professor told us to simplify our design by removing the class equipment table. He also suggested a redesign of the class registrations table by adding a primary key to it, and adding registration date and class type attributes to the table. Also, in our project 1 final draft in the rubric, we were told that we need to move the project title, team name, and members to the top of the document.

7. From Hayden Barnes: Our project looks good and correct. Our project meets all requirements and doesn't need updating.

8. From Aspen Freeze: The ERD, schema, and outline match each other. We are missing the camelcase requirement because classregID should be classregID. We should add cascade operations. We should add more comments to make the document clear. We need to clean up and modify our SQL code to make it easier to import. We should think about potential 3NF issues and fix our database to accommodate them. The intersection table is done well and so is most of the SQL. Keys are done well. Double check that our lines should be dashed instead of solid.

9. From Alexander Lane: It doesn't make sense that class only has one piece of equipment. Data could be 3NF but is definitely 2NF. SQL didn't run so it needs to be fixed and cleaned up. The document, tables, outline and diagrams look good and are consistent.

10. From the professor: Good job on the project. Use subqueries in SQL instead of hardcoding. Add cascade methods to prevent delete/update anomalies. Clean up SQL code. Make the ERD Diagram contain more information, so it is more helpful to our database managers.

11. From the professor: We need to update the ERD diagram so it doesn't look exactly like the Schema diagram and that we should look at the ERD vs Schema Diagram in the modules to see how to modify the document. We implemented CRUD operations which was good, but we will need to remove what we did because we have to implement our CRUD functionality using PL/SQL. We should be transparent about if we used AI or not. Also we no use Select * in our DML because it may cause unexpected issues.
12. Chase Schrepel:The project website looks good and meets all the requirements, aside from one thing. The one thing is that we might get possible delete anomalies like Samantha Hoffman mentioned.

13. Jonathan Salfen: The website looks good and meets all the requirements.

14. Benjamin Lynch: The website looks good and meets the draft requirements. For our SQL files we should use select to select specific rows rather than using SELECT *. We should remove the trainerID and equipment ID from the display of classes.

15. Samantha Hoffmann: The website looks good and meets requirements. We should look out for delete anomalies. We change the classes table so that the equipmentID section can be NULL to prevent delete anomalies and support people bringing their own equipment. The program should throw an error if class gets deleted because equipment was removed from it.

16. Professor Curry: Our work and website are good, but they have a few issues. There are no NEW or Update forms in the UI, possibly because they were removed during the making of our final draft. The DDL and DML have good comments and were updated well. We should look at how to cite AI if used in our project. We need to fix our ERD diagram. We should figure the purpose of displaying the intersection table and if we do display it, it should include classType. We should check to see if we are doing more than is required for the project. We should make sure our project meets the project requirements before adding extra stuff. If we have questions

we should message the professor on teams. With continued effort we should be able to address all issues and be successful on the final project and in the next project step.

17. Lucas Feldsien: Everything works and the project and the UI look good.

18. Collin Fugate: The tables have the right functionality and are working, when showing foreign keys we should use names instead of IDs in our UI. The rest works, but we need to set the foreign key checks to 0 when dropping tables. The delete works. The members' cards link to trainers and need to be fixed. The edit/add forms don't have post routes yet and need them. We need to set the WHERE criteria in the DML to WHERE ID =? To match the formatting of apps.js file. For the UI we should make it so users don't have to enter IDs in forms and let the database handle that. Good progress was made.

19. Anthony Giusti Shindelus: CRUD functionality works and the UI looks good. We should display things which are referenced by foreign by displaying their names instead of their IDs in the UI.

20. Nishat Yasmin: The select operations work. The reset functionality works. The delete functionality works. We should check to make sure that our on UPDATE CASCADE is needed. The UI looks good, but could use improvement. We should display foreign key names instead of IDs. We should add a category table, or the option for category in equipment edit and add forms via dropdown instead of a text field. Make specialty a separate table, or the option for specialties be a dropdown menu in trainer edit and add forms, instead of a text field. Add a confirmation pop up for resetting the database. Make the trainer table have the edit and delete buttons stacked up on top of each other instead of next to each other. We made overall good progress and had a few small mistakes we could fix. We should explain why our database is 3NF.

21. Professor Curry: We have made good progress. The delete and reset work. We addressed the comments about foreign keys and ERD diagrams. The edit doesn't work yet and is not expected to work yet. We are on track to get the project done on time. We should keep up the good work.

Actions based on feedback:

- Added to the overview statement more details on the problems faced under the previous analog gym system
- Briefly noted a goal of creating a framework for future growth
- Mentioned a class capacity of 15 people, added the capacity attribute to the Classes table
- Mentioned class constraints at the end of the overview
- Standardized the descriptions of 1:M, M:1, and M:M relationships
- Clarified composite primary keys in ClassEquipment and ClassRegistration tables
- Fixed LastName in the members table in the ERD Diagram and made it into lastName
- Added project Title
- Removed classequipment table and replaced the M:N relationship between classes and equipment to a M:1
- Added foreign key to classes called equipmentID to facilitate M:1 relationship between classes and equipment
- Added a primary key to ClassRegistrations called classregID
- Added classType and registrationDate attributes to the ClassRegistrations Table
- Moved team name and members to the top of the project
- Moved project title to top of the document
- Cleaned up SQL code and comments in SQL DDL file
- Added on cascade methods into the DDL file
- Added more information to the ERD diagram to make it more descriptive and helpful
- Made classregID into classRegID in outline, ERD diagram, schema diagram, and DDL file so the database maintains camelcase requirements.

- Modified ERD to not be the same as the schema diagram and so it only contains primary keys and attributes which we thought were relevant to the project description.
- Modified DML file to longer use SELECT *, it now just selects all the columns in each table
- Removed database update and delete queries.
- Added AI citations to source code
- Re-added update and delete and insert forms
- Added classType, memberID, and registrationDate attributes to ClassRegistrations table in ERD Diagram
- Added a table of contents to better organize draft pdf
- Fix the members card on the homepage to link to the members page instead of the trainers page
- Switched WHERE criteria in update and deletes queries to be WHERE ID = ?
- Remove ID type entry from forms for primary key IDs
- Made foreign keys to show by their name in the UI instead of their ID
- Made the edit and delete button in Trainers be stacked on top of each other instead of next to each other
- Made foreign key entry in forms be dropdown instead of text entry

Fixes in Feedback We Chose to Ignore:

- We chose not to make our database 3NF because our database already meets the project requirements of 2NF. Trying to normalize the database to 3NF could cause unnecessary problems in trying to update the database that we feel are not worth our time and effort.
- Why we chose to keep a 1:M between equipment and classes: The professor suggested simplifying our database in his feedback on our step 1 draft by getting rid of the many to

many relationship between classes and equipment. We simplified the relationship between classes and equipment to M:1 based on the professor's feedback. We are gonna keep the 1:M relationship to make sure we keep our database easy to implement and follow the professor's advice.

- We chose to keep equipmentID fields in the Classes table as having not null criteria to ensure that all classes are assigned gym-provided equipment. This simplifies management by maintaining standard equipment availability rather than relying on members to bring their own equipment.
- We chose to keep our website at five entities instead of four because we believe it will keep our website with its intended purpose and we believe we can properly implement all five entities. Having five entities better fits our database design than four entities
- We chose not to add dropdowns or make separate tables for the category attribute in equipment and the specialty in trainers because we felt that it would be too much work to add another entity and implement it. We also think that text entries for these attributes better fit the database because they allow new specialties or categories to be added to the database for an equipment or trainer easier than having a dropdown menu. If we want to add a new category or specialty and the form entry for these attributes was a dropdown then we would have to change our code to add new things and then select that new option. If we use a text field we only have to enter text, not change our code and select a new option. Also having a text field allows for multiple categories and specialities to be added to an equipment or trainer easier than having a dropdown.

- We opted not to include a confirmation prompt for resetting the database because the function is already made explicit through the UI design. Implementing a prompt was viewed as an unnecessary feature that would add friction to the administrative work.

- We chose to keep on updating cascade just in case an error occurs where it is necessary and the professor encouraged us to use it. Also keeping the code this way doesn’t cause any problems so it seemed like an unnecessary change.

- We opted not to add foreign key checks during the reset process because the manual DROP TABLE sequence is already sufficient. Since the restoration script is structured to drop and recreate tables in a specific, consistent order, foreign key constraints are naturally respected.